

EXPERIMENT 1: 8 PUZZLE PROBLEM

Aim

To develop a Python program to solve the 8-Puzzle problem using an AI search technique.

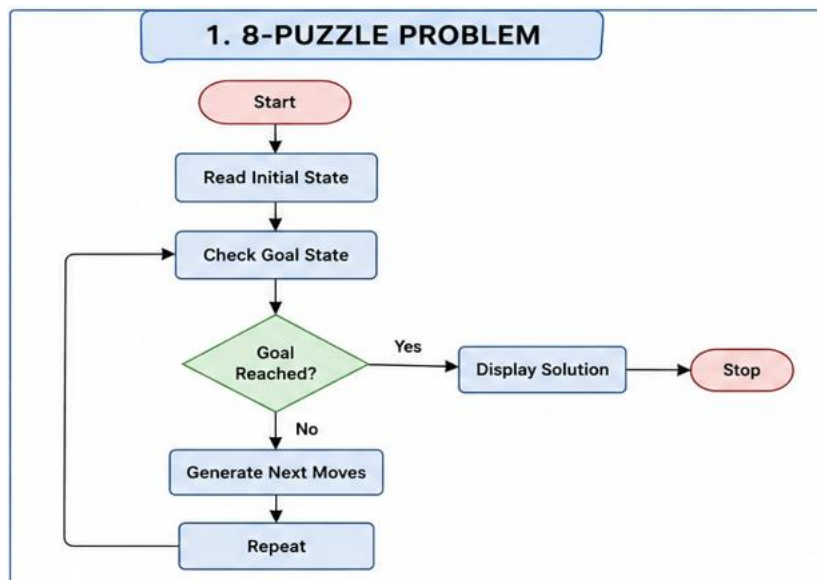
Objective

- To understand state-space search.
- To find the shortest path from the initial state to the goal state.

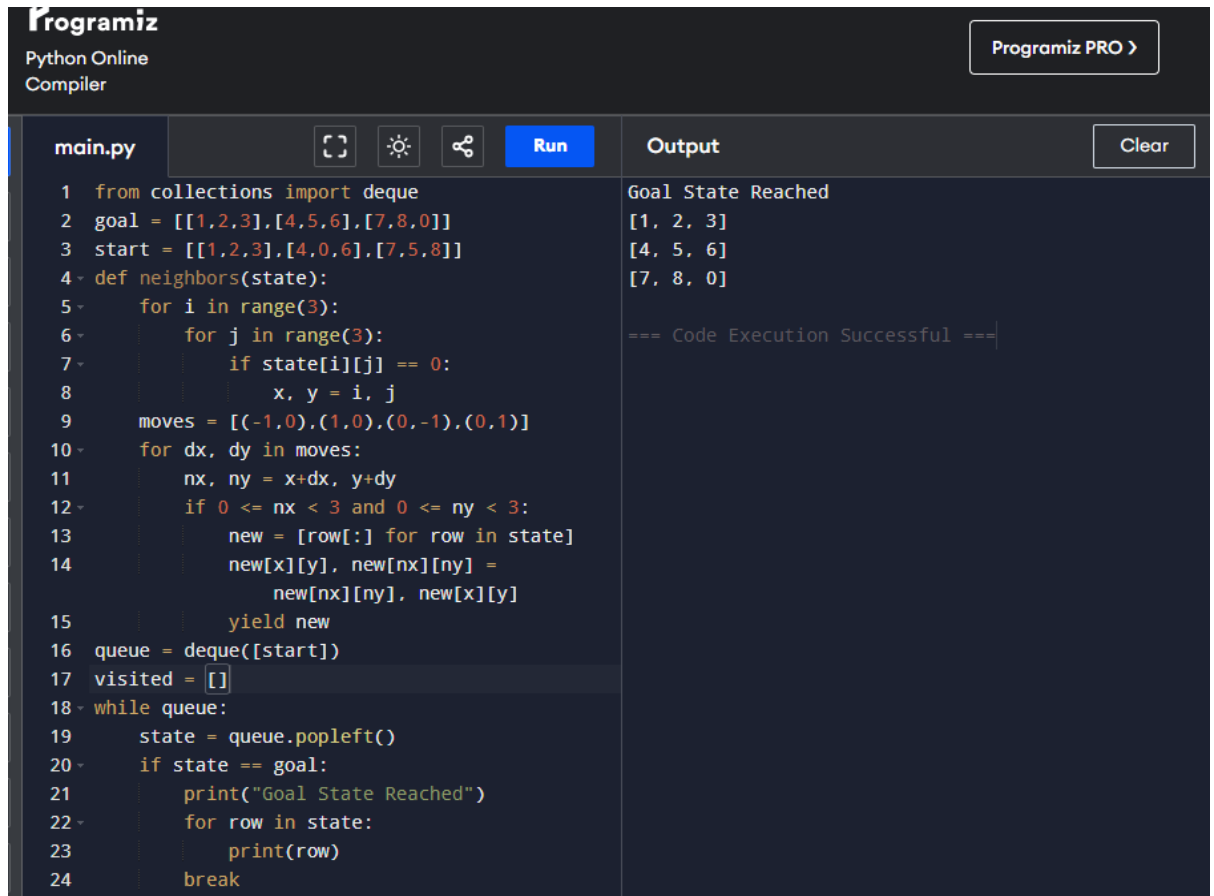
Algorithm

1. Start.
2. Read the initial puzzle state.
3. Compare it with the goal state.
4. If the goal is reached, display the solution.
5. Otherwise, generate all possible moves of the blank tile.
6. Select the next valid state.
7. Repeat Steps 3–6 until the goal state is reached.
8. Stop.

Flowchart



OUTPUT SCREENSHOT



The screenshot displays the Programiz Python Online Compiler interface. The editor on the left contains a Python script for solving the 8-Puzzle problem using a breadth-first search algorithm. The script defines a goal state, a start state, and a neighbors function. It uses a deque to manage the search queue and a visited set to avoid revisiting states. The output on the right shows the goal state reached and the final state of the puzzle.

```
1 from collections import deque
2 goal = [[1,2,3],[4,5,6],[7,8,0]]
3 start = [[1,2,3],[4,0,6],[7,5,8]]
4 def neighbors(state):
5     for i in range(3):
6         for j in range(3):
7             if state[i][j] == 0:
8                 x, y = i, j
9             moves = [(-1,0),(1,0),(0,-1),(0,1)]
10        for dx, dy in moves:
11            nx, ny = x+dx, y+dy
12            if 0 <= nx < 3 and 0 <= ny < 3:
13                new = [row[:] for row in state]
14                new[x][y], new[nx][ny] = new[nx][ny], new[x][y]
15                yield new
16 queue = deque([start])
17 visited = []
18 while queue:
19     state = queue.popleft()
20     if state == goal:
21         print("Goal State Reached")
22         for row in state:
23             print(row)
24         break
```

Goal State Reached
[1, 2, 3]
[4, 5, 6]
[7, 8, 0]

=== Code Execution Successful ===

Result

Thus, the 8-Puzzle problem was solved successfully, and the goal state was reached.

Conclusion

Thus, the 8-Puzzle problem was solved successfully by reaching the goal state using a search technique.

Experiment 2: 8-Queen Problem

Aim

To write a Python program to solve the 8-Queen problem using the Backtracking algorithm.

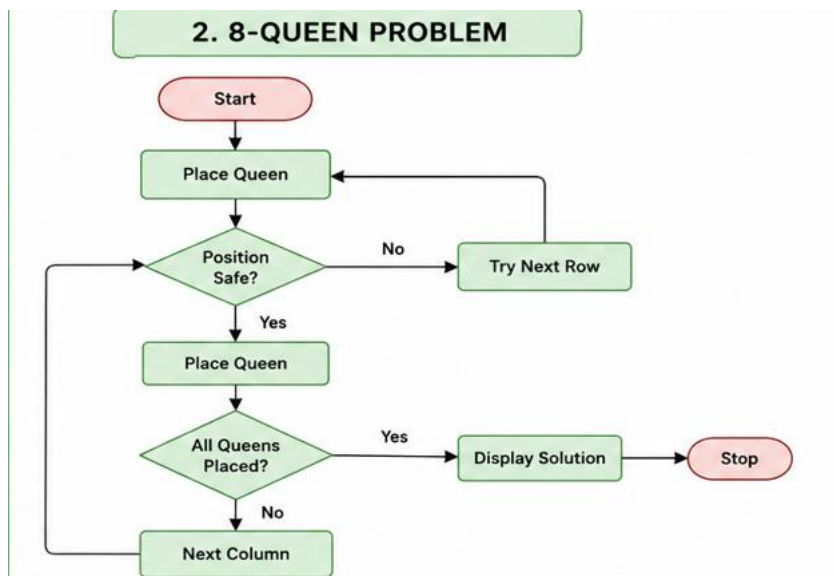
Objective

- To place eight queens on a chessboard.
- To ensure that no two queens attack each other.

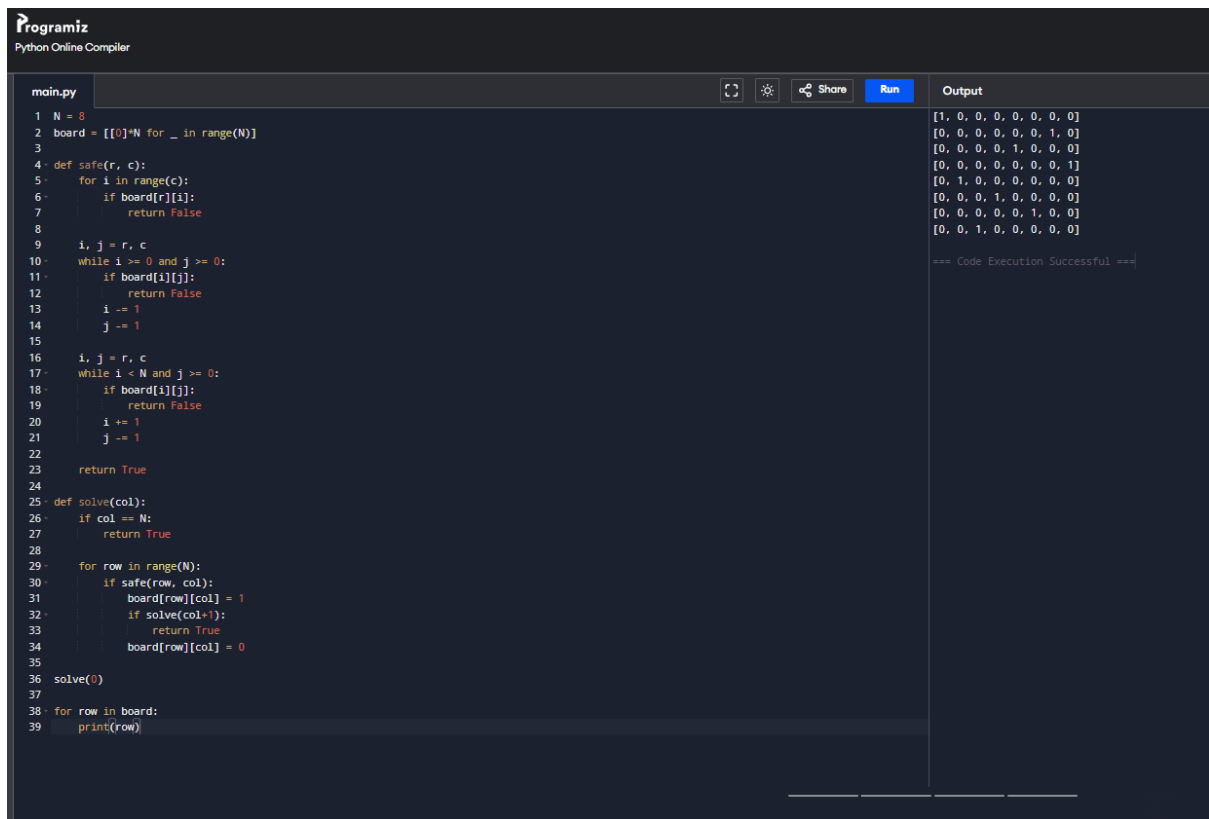
Algorithm

1. Start.
2. Place a queen in the current column.
3. Check whether the position is safe.
4. If safe, place the queen and move to the next column.
5. If not safe, try the next row.
6. Repeat until all queens are placed.
7. Display the solution.
8. Stop.

Flowchart



OUTPUT SCREENSHOT



```
main.py
1 N = 8
2 board = [[0]*N for _ in range(N)]
3
4 def safe(r, c):
5     for i in range(c):
6         if board[r][i]:
7             return False
8
9     i, j = r, c
10    while i >= 0 and j >= 0:
11        if board[i][j]:
12            return False
13        i -= 1
14        j -= 1
15
16    i, j = r, c
17    while i < N and j >= 0:
18        if board[i][j]:
19            return False
20        i += 1
21        j -= 1
22
23    return True
24
25 def solve(col):
26     if col == N:
27         return True
28
29     for row in range(N):
30         if safe(row, col):
31             board[row][col] = 1
32             if solve(col+1):
33                 return True
34             board[row][col] = 0
35
36 solve(0)
37
38 for row in board:
39     print(row)
```

```
[1, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 1, 0]
[0, 0, 0, 0, 1, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 1]
[0, 1, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 1, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 1, 0, 0]
[0, 0, 1, 0, 0, 0, 0, 0]

=== Code Execution Successful ===
```

Result

Thus, the 8-Queen problem was solved successfully by placing all eight queens without attacking each other.

Conclusion

Thus, the Backtracking algorithm successfully solved the 8-Queen problem.

Experiment 3: Water Jug Problem

Aim

To write a Python program to solve the Water Jug problem.

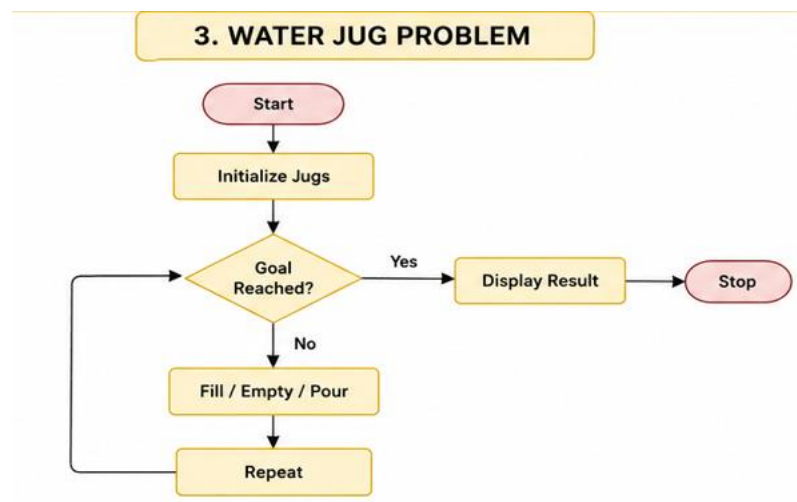
Objective

- To obtain the required quantity of water.
- To understand state-space search.

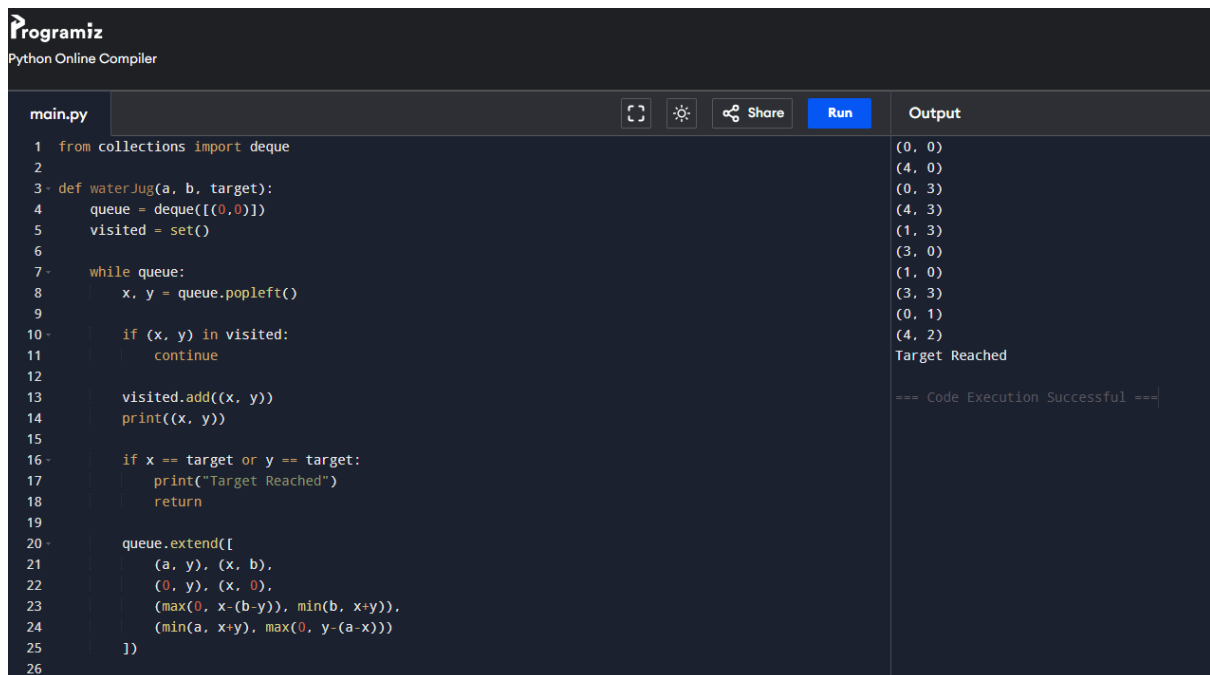
Algorithm

1. Start.
2. Initialize both jugs.
3. Perform fill, empty, or pour operations.
4. Check whether the target amount is reached.
5. If yes, display the result.
6. Otherwise, repeat the operations.
7. Stop.

Flowchart



OUTPUT SCREENSHOT



The screenshot shows the Programiz Python Online Compiler interface. The code editor on the left contains a Python script for solving the water jug problem using a breadth-first search algorithm. The script defines a function `waterJug(a, b, target)` that uses a queue and a visited set to find the shortest path to the target state. The output panel on the right displays the sequence of states visited during the search, starting from (0, 0) and ending at the target state (4, 2). The output also includes a confirmation message: "=== Code Execution Successful ===".

```
1 from collections import deque
2
3 def waterJug(a, b, target):
4     queue = deque([(0,0)])
5     visited = set()
6
7     while queue:
8         x, y = queue.popleft()
9
10        if (x, y) in visited:
11            continue
12
13        visited.add((x, y))
14        print((x, y))
15
16        if x == target or y == target:
17            print("Target Reached")
18            return
19
20        queue.extend([
21            (a, y), (x, b),
22            (0, y), (x, 0),
23            (max(0, x-(b-y)), min(b, x+y)),
24            (min(a, x+y), max(0, y-(a-x)))
25        ])
26
```

Output:

```
(0, 0)
(4, 0)
(0, 3)
(4, 3)
(1, 3)
(3, 0)
(1, 0)
(3, 3)
(0, 1)
(4, 2)
Target Reached

=== Code Execution Successful ===
```

Result

Thus, the required quantity of water was obtained successfully.

Conclusion

Thus, the Water Jug problem was solved successfully using state-space search.

Experiment 4: Crypt-Arithmetic Problem

Aim

To write a Python program to solve the Crypt-Arithmetic problem.

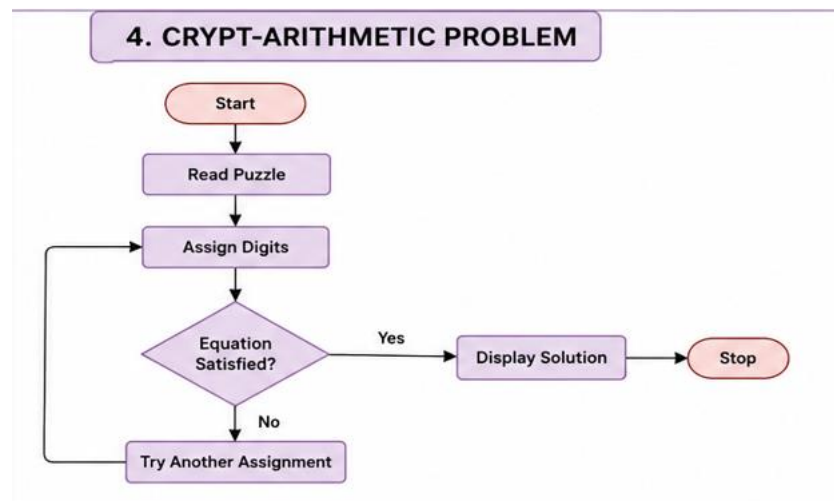
Objective

- To assign unique digits to letters.
- To satisfy the given arithmetic equation.

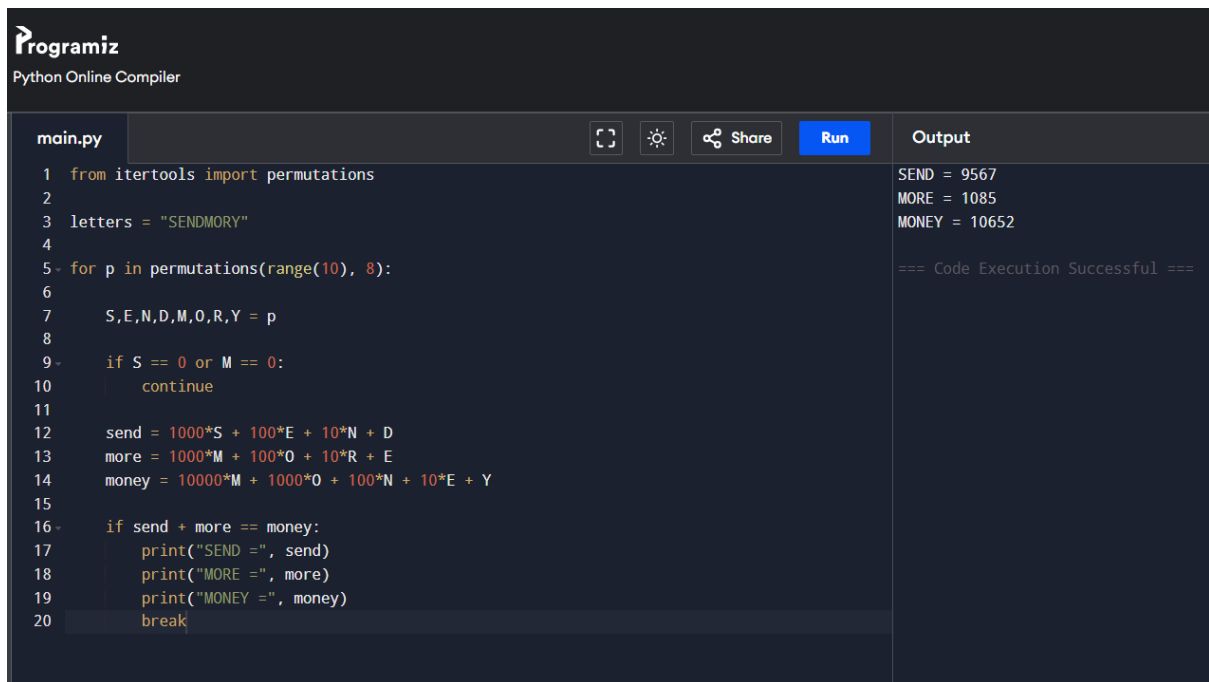
Algorithm

1. Start.
2. Read the puzzle.
3. Assign digits to the letters.
4. Check whether the equation is satisfied.
5. If yes, display the solution.
6. Otherwise, try another assignment.
7. Stop.

Flowchart



OUTPUT SCREENSHOT



The screenshot shows the Programiz Python Online Compiler interface. The code in `main.py` is as follows:

```
1 from itertools import permutations
2
3 letters = "SENDMORY"
4
5 for p in permutations(range(10), 8):
6
7     S,E,N,D,M,O,R,Y = p
8
9     if S == 0 or M == 0:
10         continue
11
12     send = 1000*S + 100*E + 10*N + D
13     more = 1000*M + 100*O + 10*R + E
14     money = 10000*M + 1000*O + 100*N + 10*E + Y
15
16     if send + more == money:
17         print("SEND =", send)
18         print("MORE =", more)
19         print("MONEY =", money)
20         break
```

The output on the right shows the results of the successful execution:

```
SEND = 9567
MORE = 1085
MONEY = 10652

=== Code Execution Successful ===
```

Result

Thus, the Crypt-Arithmetic problem was solved successfully by assigning unique digits to the letters.

Conclusion

Thus, the Crypt-Arithmetic problem was solved successfully, satisfying the equation **SEND + MORE = MONEY**.